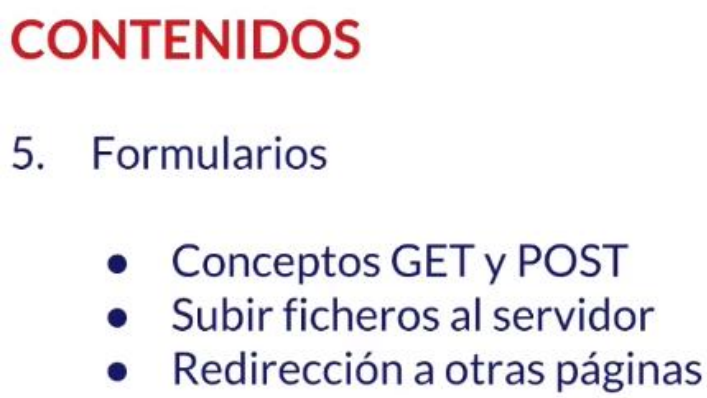
41 Conceptos GET y POST.

Aquí trabajan conjuntamente html y php.



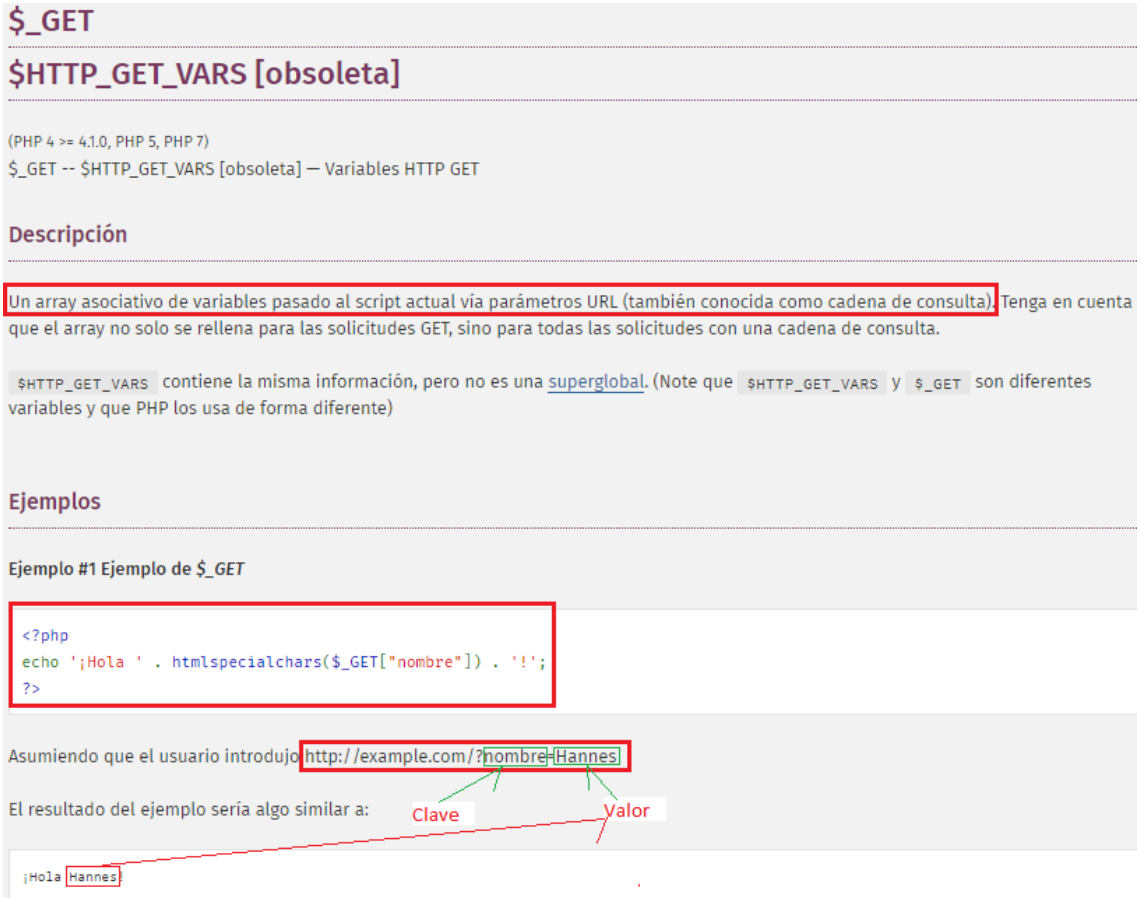
A través de los formularios se pueden enviar datos al servidor para que este los proceso.

Ahora se va a trabajar con una página html que envía datos y un script o fichero php que recibe esos datos y los procesa en el servidor.

GET.

Documentación oficial:

https://www.php.net/manual/es/reserved.variables.get.php



$_GET

$HTTP_GET_VARS [obsoleta]

(PHP 4 >= 4.1.0, PHP 5, PHP 7)
$_GET -- $HTTP_GET_VARS [obsoleta] — Variables HTTP GET

Descripción

Un array asociativo de variables pasado al script actual vía parámetros URL (también conocida como cadena de consulta). Tenga en cuenta que el array no solo se rellena para las solicitudes GET, sino para todas las solicitudes con una cadena de consulta.

`$HTTP_GET_VARS` contiene la misma información, pero no es una superglobal. (Note que `$HTTP_GET_VARS` y `$_GET` son diferentes variables y que PHP los usa de forma diferente)

Ejemplos

Ejemplo #1 Ejemplo de *$_GET*

```
<?php
echo '¡Hola ' . htmlspecialchars($_GET["nombre"]) . '!';
?>
```

Asumiendo que el usuario introdujo http://example.com/?nombre=Hannes

El resultado del ejemplo sería algo similar a:

Clave

Valor

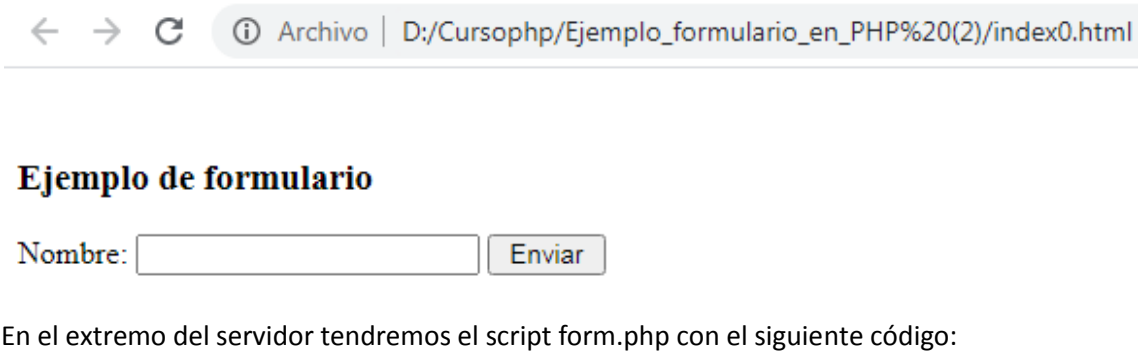
¡Hola Hannes

Envía datos al servidor usando la barra de direcciones del navegador y, por lo tanto, esos datos enviados se visualizan en la barra de direcciones. Por esta forma de proceder No es muy seguro. Los datos enviados constituyen un array asociativo que almacena información en formato clave/valor: la clave funciona como índice del array y el contenido de esa casilla del array es el valor.

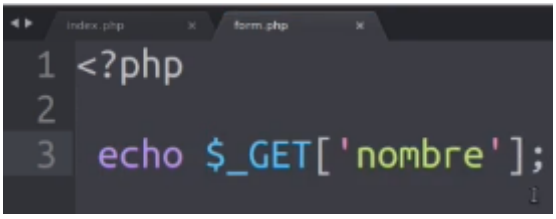
Creamos un formulario en html para enviar datos al servidor:



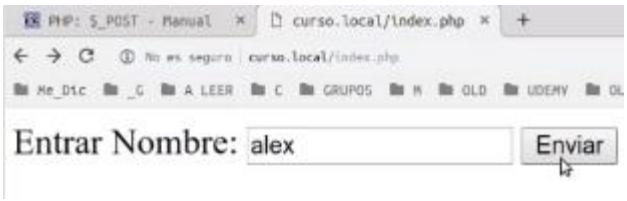
Si lo visualizamos con un navegador:



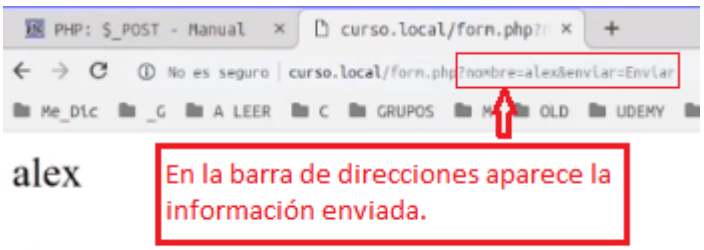
En el extremo del servidor tendremos el script form.php con el siguiente código:



Si introducimos alex:



Y pulsamos enviar, se ejecutará el script, leerá en nombre enviado y lo escribirá:



En la barra de direcciones aparece:

/form.php?nombre=alex&...

Clave

Valor

Aquí "nombre" sería la clave y "alex" el valor para esa clave. Estos datos los meten internamente en un array asociativo al que se puede acceder en el extremo del servidor donde está el script php que atenderá esta petición. La forma de acceder a ese array usando envío GET sería:

$_GET['nombre']

Clave

Y esa instrucción devolvería:

"alex"

Valor

Por lo tanto, debemos gestionar un array asociativo superglobal (se crean y rellenan en un extremo y se usan en el otro extremo) en el que los índices o claves son el nombre de la variable que queremos recibir en el lado del servidor.

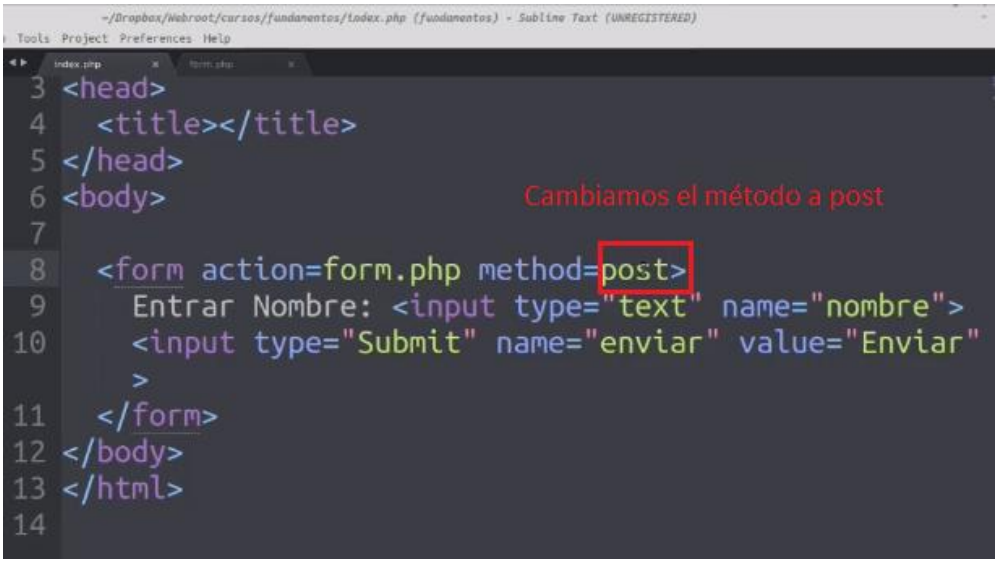
El hecho de que $_GET sea superglobal quiere decir que lo puedes usar en cualquier parte de tu código, incluso dentro de funciones, sin necesidad de declarar que vas a usarlo con:

global $variable;

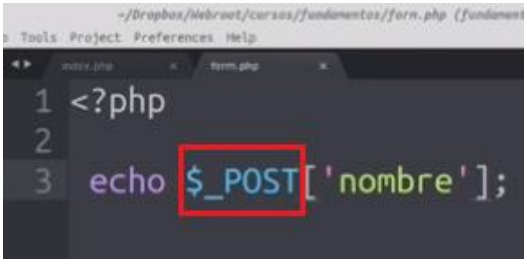
Con el método GET, en la barra de direcciones aparece la información enviada. No es seguro. Además, el uso de GET tiene bastantes limitaciones: no se pueden enviar datos binarios, el número de caracteres que se pueden enviar está limitado, hay caracteres especiales que no se pueden usar directamente en la URL (ejemplo espacios en blanco) etc.

POST

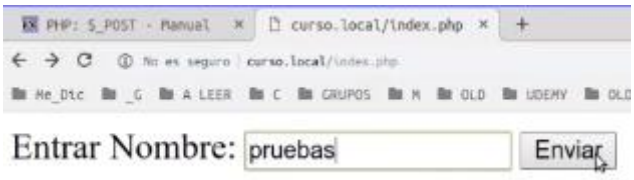
El método POST no tiene las limitaciones que el GET y no envía su información por la barra de navegación. Para usarlo, cambiamos el método a POST:



Y en la parte que se ejecuta en el servidor, para nuestro ejemplo “form.php” se recibe así:



Y se obtendrá como salida:



Y al pulsar en el botón “Enviar”, obtendremos:

